

Análisis de impacto cruzado y opciones de rediseño

Paquete `org.openmarkov.core.model.network.potential`

Continuación del análisis previo *rediseño-potenciales*

Manuel Arias Calleja

Mayo de 2026

Índice

1. Contexto y alcance	3
1.1. Documento previo y diferencias con éste	3
1.2. Inventario rápido del paquete	3
1.3. Jerarquía actual	4
2. Metodología del análisis cruzado	4
3. Hallazgos del análisis cruzado	5
3.1. Subclases externas de <code>Potential</code>	5
3.2. Accesos directos a campos públicos	5
3.3. Verificaciones <code>instanceof</code> contra subtipos	6
3.4. Uso de las fachadas <code>*PotentialOperations</code>	8
3.5. Importaciones directas de <i>internals</i> de <code>operation/</code>	8
3.6. Sistema de plugins en torno a <code>Potential</code>	8
3.7. Constructores invocados externamente	9
3.8. Métodos públicos de <code>Potential</code> invocados desde fuera	9
3.9. Mutadores específicos por subtipo	10
3.10. Serialización PGMX en detalle	10
3.11. GUI: ediciones, paneles y mutaciones	11
3.12. Tests externos	11
3.13. Resumen agregado por módulo	12
4. Replanteamiento de las opciones de refactor	12
4.1. Opción A — Higiene local	12
4.2. Opción B — Completar la migración a interfaces de capacidad	13
4.3. Opción C — Renombrar <code>PotentialRole.UNSPECIFIED</code> a <code>UTILITY</code>	14
4.4. Opción D — Aplanar la fachada de operaciones	15
4.5. Opción E — Unificar las firmas de <code>tableProject</code>	15
4.6. Opción F — Composición sobre herencia (<code>Strategic / Uncertain</code> como decoradores)	15
4.7. Opción G — Inmutabilidad estricta	16
4.8. Opción H — Sustituir cadena <code>instanceof</code> de PGMX por <i>strategy/visitor</i>	16
4.9. Comparativa de opciones	17
5. Plan de acción por fases	17
5.1. Fase 0 — Anclar el comportamiento actual con <code>properties</code>	17
5.2. Fase 1 — Higiene local (Opción A)	18
5.3. Fase 2 — Capacidades canónicas (Opción B + Opción E)	18

5.4. Fase 3 — Strategy en PGMXWriter (Opción H)	18
5.5. Fase 4 — Aplanar fachadas (Opción D)	19
5.6. Fase 5 — Limpieza semántica de PotentialRole (Opción C, opcional)	19
5.7. Fases postergables: F (composición) y G (inmutabilidad)	19
6. Riesgos y mitigaciones	20
6.1. Riesgos transversales	20
6.2. Riesgos por fase	20
7. Recomendación final	20
A. Comandos usados para el análisis cruzado	21
B. Glosario	21

Resumen ejecutivo

Este documento extiende el análisis previo del paquete `potential` (*rediseño-potenciales*, marzo de 2026) con un **mapa exhaustivo del acoplamiento externo**: qué módulos usan qué clases, métodos y campos del paquete, y dónde rompería cada decisión de refactor. El recuento principal:

- **372** archivos `.java` importan algo del paquete; **317** referencian alguna variante de `TablePotential`.
- **0** subclases externas de cualquier subtipo de `Potential`. Toda la jerarquía de potenciales vive dentro del paquete.
- **14** `@PotentialPanelPlugin` en `org.openmarkov.gui` que mencionan clases concretas de potencial por literal de clase.
- **315** accesos externos a `getValues()[i]` (lectura), de los cuales **7** son escrituras directas en producción.
- **63** invocaciones externas a métodos mutadores específicos de potenciales (`setValues`, `setUncertainValues`, `setNoisyParameters`, `setLeakyParameters`, `setBranches`, `setRootVariable...`).
- Cadena `instanceof` de **14** subtipos en `PGMXWriter_0_2.java:746-860`: punto único que hay que tocar para cada nuevo potencial.

A la luz de estos datos, la recomendación se confirma: las opciones de *Nivel 1* (higiene local) y *Nivel 2* (consolidación de capacidades) son seguras y rentables; las de *Nivel 3* (composición y/o inmutabilidad estricta) están condicionadas por puntos concretos de mutación y por el contrato de los plugins GUI/PGMX. El plan de acción se estructura en cinco fases incrementales (5), cada una con criterios de aceptación y riesgo evaluado contra los datos de impacto de la sección 3.

1. Contexto y alcance

1.1. Documento previo y diferencias con éste

En `org.openmarkov/docs/rediseño-potenciales/` ya existe un análisis de diseño del paquete (marzo de 2026), centrado en problemas *intra*-paquete (triple responsabilidad de `TablePotential`, jerarquía paralela `StrategicTablePotential/UncertainTablePotential`, abuso de `PotentialRole`, etc.). Ese trabajo culminó con la extracción de las interfaces de capacidad (`Projectable`, `Reorderable`, `Scalable`, `StrategyCarrier`, `UncertaintyCarrier`, `CEUtilityPotential`) y la separación parcial de `StrategicTablePotential` y `UncertainTablePotential`.

Este documento responde a la pregunta complementaria: **¿qué impacto tiene cada paso adicional de rediseño en el resto del workspace?** Por tanto:

- No vuelve a discutir los problemas internos ya catalogados; los *asume*.
- Cataloga el acoplamiento externo con datos cuantitativos (`file:line`) recogidos sobre el árbol vivo en `/home/manuel/workspace-openmarkov/`.
- Reformula las opciones de refactor en función de su huella en módulos externos.
- Propone un plan de acción por fases con criterios verificables.

1.2. Inventario rápido del paquete

Recordatorio breve, sin entrar en detalle (ver el documento previo):

- **Base abstracta:** `Potential` (735 líneas) y `AbstractIndexedPotential`.

- **Interfaces de capacidad:** Projectable, Reorderable, Scalable, StrategyCarrier, UncertaintyCarrier, CEUtilityPotential.
- **Tabla densa:** TablePotential (759 líneas), UncertainTablePotential, StrategicTablePotential, AugmentedProbTable y GTablePotential<E>.
- **Definiciones simbólicas:** UniformPotential, DeltaPotential, FunctionPotential, LinearCombinationPotential, GLMPotential, ProductPotential, SumPotential, SameAsPrevious, CycleLengthShift, ExactDistrPotential, etc.
- **Distribuciones paramétricas:** BinomialPotential, ConditionalGaussianPotential, ExponentialPotential, ExponentialHazardPotential, WeibullHazardPotential, DiscretizedCauchyPotent, UnivariateDistrPotential.
- **Subpaquetes:** canonical/ (ICI: Max, Min, Tuning, MinMax), treeadd/ (TreeADDPotential, TreeADDBranch, Threshold), sdag/ (SDAGStrategyTree), plugin/ (@PotentialType, PotentialUtils).
- **Subpaquete operation/:** ~4169 líneas en 15 ficheros. Fachadas (PotentialOperations, DiscretePotentialOperations), implementación interna (TablePotentialArithmetic, *Elimination, *Maximization, *Merge, *Transform), descomposiciones de utilidad (Marginalization, SumOutVariable, MaxOutVariable) y auxiliares.

1.3. Jerarquía actual

La figura 1 resume la jerarquía tras el primer rediseño. Las interfaces de capacidad (azul) ya existen pero su adopción es parcial: la base Potential sigue ofreciendo *defaults* para tableProject, reorder y scalePotential que lanzan UnsupportedOperationException.

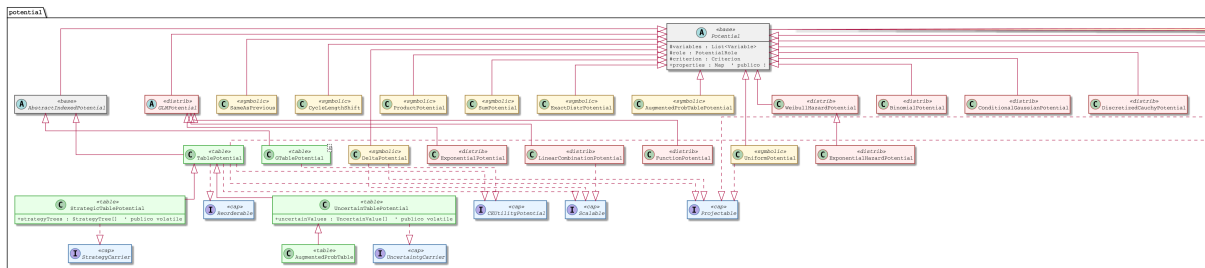


Figura 1: Jerarquía actual del paquete potential (post Rediseño 1). En verde, los *table-potentials*; en amarillo, los simbólicos; en rojo claro, las distribuciones paramétricas; en azul, las interfaces de capacidad.

2. Metodología del análisis cruzado

Los datos cuantitativos provienen de búsquedas en el árbol /home/manuel/workspace-openmarkov/ con las exclusiones siguientes:

- Se excluye target/, build/, out/ y cualquier artefacto generado.
- Se excluye el propio paquete org.openmarkov.core/.../potential/ (sólo *usuarios externos*).
- Las búsquedas distinguen producción de tests (src/main/ vs src/test/) cuando es relevante.
- Cuando un patrón aparece muchas veces, se da el conteo agregado más 2–5 ejemplos representativos con file:line exacto.

Las cifras son verificables con los comandos citados en cada subsección. Donde el primer barrido del agente automático difería del barrido directo (p.ej. accesos a `.strategyTrees`), se reporta el dato *verificado* manualmente y se anota la discrepancia.

3. Hallazgos del análisis cruzado

3.1. Subclases externas de `Potential`

Hallazgo: 0 subclases externas de cualquier subtipo de `Potential`.

Todas las apariciones de `extends Potential|TablePotential|...` en el workspace están dentro del propio paquete. Las menciones externas a `Potential` son siempre de la forma:

- Parámetro de tipo genérico: `Map<Class<? extends Potential>, ...>` en `PGMXReader_0_2.java:60,72` y `PotentialPanelManager.java:35`.
- Variables de tipo `Class<? extends Potential>` para reflexión (`PotentialEditDialog.java:212,469`).
- Wildcards en colecciones: `List<? extends Potential>` en `ProbNet`, `ProbNetPotentialQueries`, `ProbNetTest`.

Implicación clave: *el sistema de plugins documentado (`@PotentialType + classgraph`) no se usa por ningún tercero identificable en este árbol*. Un refactor que cambie las clases base no necesita una capa de compatibilidad para subclases ajenas. La rotura sería local al paquete y a sus consumidores tipados.

3.2. Accesos directos a campos públicos

Los campos potencialmente expuestos son `UncertainTablePotential.uncertainValues` (`public volatile`), `StrategicTablePotential.strategyTrees` (`public volatile`) y `Potential.properties` (`public`).

uncertainValues

11 accesos externos en producción y tests (filtrando duplicados con campos homónimos en CEP y `CostEffectivenessException`, que no son del paquete `potential`):

- `org.openmarkov.core/.../ProbNetOperations.java:618,619,633,634` — lectura/escritura en proyección de incertidumbre (**producción, dentro de core pero fuera del paquete**).
- `org.openmarkov.core/.../modelUncertainty/SystematicSampling.java:150,162` — escritura: `uncertainNew.uncertainValues = new UncertainValue[...]` y `uncertainNew.uncertainValues[newP = ...]`.
- `org.openmarkov.core/src/test/.../SensitivityAnalysisFactory.java:58,65,129,165,202` — escrituras directas de literales en *tests*.

strategyTrees

3 accesos externos:

- `org.openmarkov.inference/.../VEOptimalIntervention.java:85,86` — **escritura directa** desde inferencia: `strategic.strategyTrees = new StrategyTree[1]; strategic.strategyTrees[0] = auxST;`.
- `org.openmarkov.integrationtests/.../inference/heuristics/Tools.java:70` — lectura vía pattern matching.

(Las apariciones en CEP.java y CostEffectivenessException.java son de campos *distintos* con el mismo nombre, no del campo de StrategicTablePotential.)

properties

0 accesos externos. Aunque el campo es public, ningún módulo externo lo lee o escribe directamente.

getValues()[i] (semánticamente equivalente a campo público)

TablePotential.getValues() devuelve la referencia viva al double[] interno. El patrón potential.getValues()[i] es ubicuo:

- **315** apariciones en código externo, distribuidas por módulo (lectura + escritura mezcladas).
- **Escrituras directas en producción** (getValues()[i] = ...):

```
1 // org.openmarkov.core/.../ProbNetOperations.java:457
2 potential.getValues()[0] = 1;
3 // org.openmarkov.core/.../ProbNetOperations.java:632
4 potential.getValues()[configIndex + i] = projectedPotential.getValues()[
    ↪ projectedConfigIndex + i];
5 // org.openmarkov.core/.../TemporalNetOperations.java:317
6 sumPotential.getValues()[j] /= 2;
7 // org.openmarkov.core/.../Variable.java:524
8 potential.getValues()[stateIndex] = 1.0;
9 // org.openmarkov.core/.../modelUncertainty/SystematicSampling.java:160
10 newPotential.getValues()[newPos] = values[j];
11 // org.openmarkov.gui/.../action/TablePotentialValueEdit.java:162
12 tablePotential.getValues()[potentialSelected] = newValue;
13 // org.openmarkov.learning.core/.../LearningAlgorithm.java:179
14 absoluteFrequencies.getValues()[j] += alpha;
```

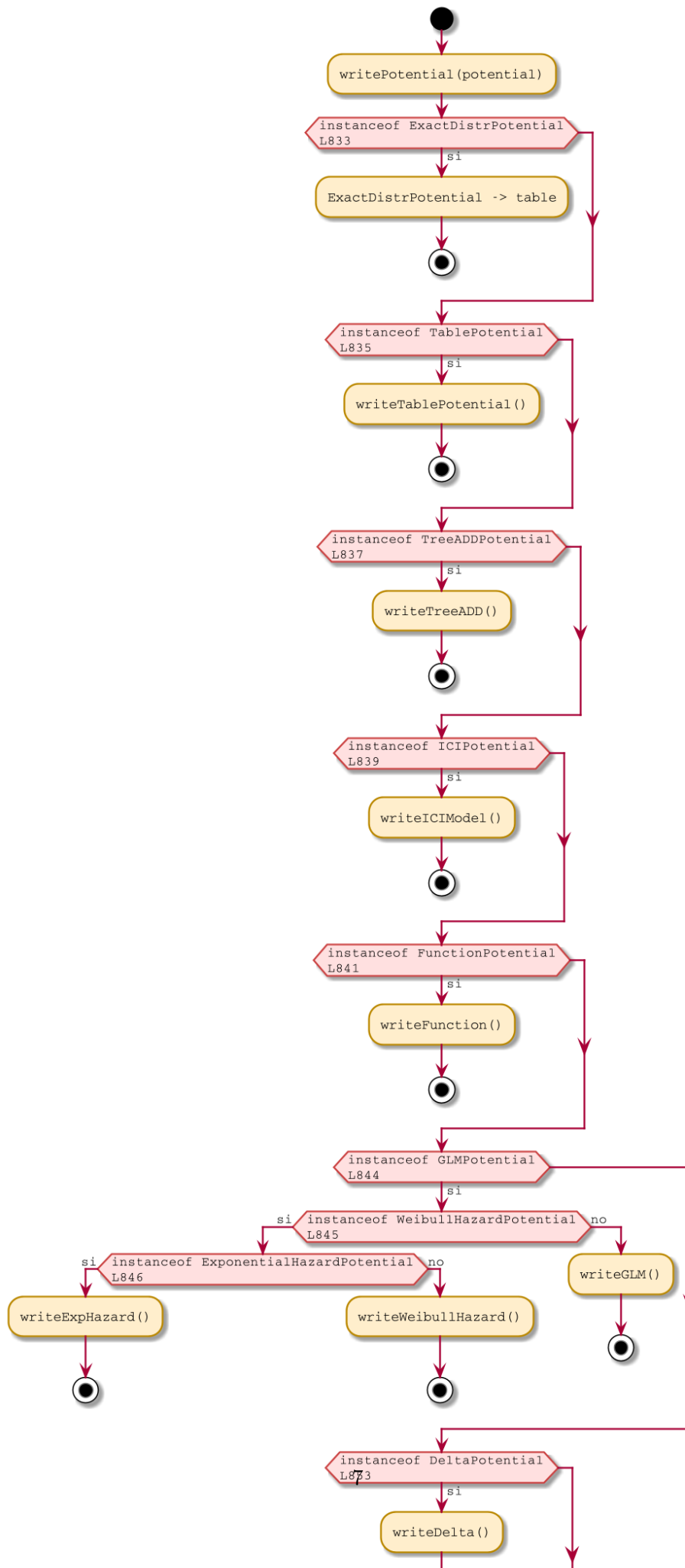
Listado 1: Escrituras directas vía getValues() en producción.

Implicación: cualquier opción que *privatice realmente el array o lo envuelva en un objeto inmutable obliga a tocar estos siete sitios y diseñar un reemplazo para todos ellos*. Las lecturas (que son la mayoría) sobreviven a un getValues() que devuelva una vista de sólo lectura.

3.3. Verificaciones instanceof contra subtipos

Las cadenas instanceof se concentran en serialización y en heurísticas de inferencia/GUI:

- **PGMX** (lectores y escritores en org.openmarkov.io): cadena exhaustiva en PGMXWriter_0_2.java:746–860 con 14 subtipos (ExactDistrPotential, TablePotential, TreeADDPotential, ICIPotential, FunctionPotential, GLMPotential, WeibullHazardPotential, ExponentialHazardPotential, DeltaPotential, BinomialPotential, AugmentedProbTablePotential, SumPotential, ProductPotential, UnivariateDistrPotential). Ver figura 2 y la tabla siguiente para línea exacta. Idéntico patrón en PGMXWriter_1_0.java:121–240.
- **Inferencia:** chequeos puntuales contra GTablePotential y TablePotential en ChanceVariableElimination, DecisionVariableElimination, DANOoperations, VEOptimalIntervention.
- **GUI:** cadenas en TablePotentialPanel, ValuesTable, PotentialEditDialog, LinkRestrictionValuesTable, PolicyTypePanel, etc.
- **Tests:** PGMXReaderWriterTest, Tools (heurísticas), InferenceProbabilityInvariantsTest.



3.4. Uso de las fachadas *PotentialOperations

DiscretePotentialOperations (453 líneas) y PotentialOperations (320 líneas) son dos fachadas con dispatch redundante hacia el subpaquete operation/. Externamente:

- DiscretePotentialOperations: 24 ficheros externos con import directo, ~80 invocaciones agregadas. Concentrado en org.openmarkov.inference (15 ficheros) y org.openmarkov.core (acciones, NodeAbsorptionHandler, Variable, etc.).
- PotentialOperations: 28 ficheros externos. Mayoritariamente en inference (14 ficheros), gui (4), core (resto), learning.core (1), integrationtests (1).

Hay solapamiento: muchos ficheros importan ambas fachadas, lo que confirma que no hay una división clara de responsabilidades entre ellas.

3.5. Importaciones directas de *internals* de operation/

10 ficheros externos saltan la fachada e importan clases internas:

```
1 inference/.../timeSliceElimination/TimeSliceElimination.java:16
2     AuxiliaryOperations
3 inference/.../dlimidevaluation/StrategyManager.java:16
4     AuxiliaryOperations
5 inference/.../variableElimination/DecisionVariableElimination.java:17
6     MaxOutVariable
7 gui/.../dialog/node/PotentialEditDialog.java:22
8     LinkRestrictionPotentialOperations
9 gui/.../dialog/common/TablePotentialPanel.java:17
10    LinkRestrictionPotentialOperations
11 gui/.../dialog/common/UnivariateDistrPotentialPanel.java:21
12    LinkRestrictionPotentialOperations
13 gui/.../dialog/common/AugmentedProbTablePotentialPanel.java:19
14    LinkRestrictionPotentialOperations
15 gui/.../component/LinkRestrictionValuesTable.java:20
16    LinkRestrictionPotentialOperations
17 core/.../model/network/NodeAbsorptionHandler.java:15
18    AuxiliaryOperations
19 core/.../action/core/SetPotentialEdit.java:12
20    LinkRestrictionPotentialOperations
```

Listado 2: Imports directos a clases internas del subpaquete operation.

Implicación: LinkRestrictionPotentialOperations ya es *de facto* parte de la API pública (5 clientes externos en producción). Cualquier reorganización de operation/ debe preservar esta clase como punto público estable, o reemplazarla por una sustitución con el mismo nombre. Lo mismo aplica para AuxiliaryOperations (3 clientes) y MaxOutVariable (1 cliente).

3.6. Sistema de plugins en torno a Potential

Hay *tres* sistemas de plugin operando en paralelo, cada uno con su propia anotación o convención (figura 3):

1. **@PotentialType(names=...)** (en core): descubrimiento por PluginSearch + classgraph, consumido por PotentialUtils (POTENTIALS_BY_NAME). Sirve para serialización XML PGMX (PGMXReader_0_2.java:1429,1445; PGMXWriter_0_2.java:673,759,906; PGMXWriter_1_0.java:121,197,199; PGMXPotentialParsers.java:145,148,151; XMLBIFReader.java:83).
2. **@ImplementationRequirements + @RequiredConstructor** (en core): contrato de construcción por reflexión. PotentialUtils.instantiateSafely (línea 86) prueba en cadena los constructores (List, CycleLength), (List, PotentialRole), (List), (SelfClass).

-
- El diagrama de flujo ilustra la implementación de la arquitectura MVC en Java, dividida en dos secciones principales: la parte de Potencial y la parte de PotencialPanel.
- Sección Potencial:**
- io (mapeo Class -> PotentialParser):**
 - PQOHPotentialParsers:**
 - parametrl() : Potential
 - usa
 - PotentialUtils.getPotentialName
 - PQOHPReader_0_2:**
 - O potentialParsers : Map<Class, PotentialParser>
 - buildPotentialParsers()
 - usa
 - usa instanciaof y
 - PotentialUtils.getClassByName
 - PQOHPWriter_0_2:**
 - cadena instanciaof
 - de 14 subtipos
 - writePotential(potential)
 - core (descubrimiento por #PotentialType):**
 - PotentialUtils:**
 - O POTENTIALS_BY_NAME : Map<String, Class>
 - getClassByType(name) : Class
 - getPotentialName(clazz) : String
 - getFilteredPotentialClasses(node)
 - instanciateSafely(clazz, vars, role, ...)
 - #PotentialType(name):**
 - escapa
 - instancia via reflexión
 - Potential (recargable):**
 - depara
- Sección PotencialPanel:**
- gui (descubrimiento por #PotentialPanelPlugin):**
 - PotentialPanel:**
 - relación de herencia con #TablePotentialPanel, #BinomialPotentialPanel, #ICIPotentialTablePanel y #PotentialPanelPlugin (potencialClasses)
 - PotentialPanelManager:**
 - O potentialPanelClassesByClass : Map<Class, Class>
 - getPotentialPanelClassOf(clazz)
 - ...11 paneles mas...**
 - #TablePotentialPanel**
 - #BinomialPotentialPanel**
 - #ICIPotentialTablePanel**
 - #PotentialPanelPlugin (potencialClasses):**
 - escanda
 - referencia por Class:**
 - Se muestra una flecha de referencia desde el método `getClassByType` de `PotentialUtils` hacia el método `getPotentialPanelClassOf` de `PotentialPanelManager`.
- Nota:** Cada nuevo subtipo de Potential obliga a actualizar 4 sitios:
1. Anotación #PotentialType
 2. Constructor que cumpla #ImplementationRequirements
 3. Cadena instanciaof en PQOHPWriter
 4. Crear #PotentialPanelPlugin propio

3.7. Constructores invocados externamente

Módulo	Inst.	Ejemplos representativos
Tests externos	194	TestNetworks.java:69, ExampleNets.java:47
org.openmarkov.inference	61	StochasticPropagation.java:201
org.openmarkov.io	34	ElviraParser.java:150,491,500,505; PGMXReader_0_2.java:1486
org.openmarkov.gui	16	ElviraWriter.java:547, PotentialEditDialog
org.openmarkov.learning.core	1	LearningAlgorithm.java

Implicación: cambios en estas firmas tienen huella muy amplia. La firma (`List<Variable>`, `PotentialRole`) es la *de facto* canónica.

Top de métodos por uso externo agregado:

Método	Ficheros	Comentario
<code>getVariables()</code>	186	ubicuo
<code>getValues()</code>	112	lectura del array
<code>getValue(. . .)</code>	55	acceso indexado
<code>setValues(double[])</code>	35	GUI, learning, tests
<code>getProbability(. . .)</code>	19	query probabilística
<code>tableProject(. . .)</code>	17	inferencia
<code>getCPT()</code>	9	expansión a tabla
<code>getUncertainValues()</code>	10	sensibilidad
<code>setUncertainValues(. . .)</code>	6	sensibilidad
<code>setPotentialRole(. . .)</code>	5	roles
<code>removeVariable(. . .)</code>	4	ediciones
<code>replaceVariable(. . .)</code>	2	ediciones
<code>addVariable(. . .)</code>	1	ediciones

Métodos sin uso externo significativo: `sampleConditionedVariable` (0), `replaceNumericVariable` (0), `scalePotential` directo (0), `reorder` directo (0), `setComment` (0), `isAdditive` (0), `hasInterventions` (0), `hasInterventionForDecision` (0). Estos son candidatos seguros a mover a interfaces de capacidad o a privatizar.

3.9. Mutadores específicos por subtipo

Llamadas externas a setters específicos de subtipo (acotadas a producción):

- `ICIPotentialEdit.java:48,50,57,59` — `setNoisyParameters(. . .)`, `setLeakyParameters(. . .)` (ediciones del modelo).
- `NodeStateEdit.java:180` — `restrictionPotential.setValues(. . .)` para link restrictions.
- `UncertainValuesEdit.java:126,127`, `UncertainValuesRemoveEdit.java:121` — `setUncertainValues(null)` y `setValues(. . .)`.
- `Choice.java:52` (inferencia) — `this.setValues(values)`.
- `DANFactory.java:1002,1009` — `setValues(double[])` sobre potenciales de restricción.

3.10. Serialización PGMX en detalle

PGMX es la fuente más sensible al rediseño. `PGMXReader_0_2`:

- `PGMXReader_0_2.java:60` — campo `public final Map<Class<? extends Potential>, PotentialParser> potentialParsers` (mapeo por Class).
- `PGMXReader_0_2.java:1429` — llamada `PotentialUtils.getNames(TablePotential.class).contains(. . .)` que condiciona la decisión sobre rol de utilidad.
- `PGMXReader_0_2.java:1445` — `PotentialUtils.getClassByName(sXmlPotentialType): lookup` dinámico de la clase a partir del XML.
- `PGMXReader_0_2.java:1449` — comentario explícito: *“Walk up the hierarchy: subclasses that inherit a parent’s @PotentialType name.”*
- `PGMXReader_0_2.java:1486` — `new TreeADDPotential(variables, topVariable, xmlRole, branches)` (instanciación directa).

`PGMXWriter_0_2`: cadena `instanceof` entre líneas 746 y 860 (ver figura 2). `PGMXWriter_1_0` replica la estrategia con un nuevo caso para `UnivariateDistrPotential` en línea 197–199 y 238.

Implicación: la serialización PGMX es un contrato de fichero. Cualquier renombrado de clase, anotación o enum (en particular `PotentialRole.UNSPECIFIED` → `UTILITY`) requiere mantener compatibilidad de *lectura* con redes existentes.

3.11. GUI: ediciones, paneles y mutaciones

Sistema de paneles

14 `PotentialPanel` concretos en `org.openmarkov.gui/.../dialog/common/` y `dialog/treeadd/`, cada uno anotado con `@PotentialPanelPlugin(potentialClasses=...)` (ver sección 3.6). `TablePotentialPanel` es el único que declara dos clases (`TablePotential.class` y `ExactDistrPotential.class`); el resto, una sola.

Ediciones GUI

En `org.openmarkov.gui/src/main/java/org/openmarkov/gui/action/`: `TablePotentialValueEdit`, `ICITablePotentialValueEdit`, `AugmentedPotentialValueEdit`, `LinkRestrictionPotentialValueEdit`. Estas son `PNEdit` subclases que aplican (y deshacen) cambios in-place sobre el potencial. La mutación en `TablePotentialValueEdit.java:162` es:

```
1 tablePotential.getValues()[potentialSelected] = newValue;
```

Listado 3: Mutación in-place vía `getValues()` en una edición.

Lectura para visualización

`ValuesTable.java:710,723,729` y `TraceTemporalEvolutionDialog.java:514,536,789,1395,1398,1473,1477` leen `getValues()[...]` para pintar tablas y trazas. `InferencePresenter.java:66` hace `individualProbabilities.get(variable).getValues()[0]` para barras de probabilidad por estado.

Implicación: envolver `getValues()` en una vista de sólo lectura es seguro para *lectura* (la mayor parte de los 315 sitios), pero el canal de mutación in-place desde la GUI requiere un punto de actualización explícito (`setValueAt(idx, double)` o `withValue(idx, double)` si vamos a inmutabilidad).

3.12. Tests externos

Tests que crean potenciales o inspeccionan su estructura interna:

- `org.openmarkov.inference/src/test/.../TestNetworks.java:69–228`, `ExampleNets.java:47–225`
— construcción de redes con constructores públicos.
- `org.openmarkov.io/src/test/.../PGMXReaderWriterTest.java:95,97,159`; `ElviraWriterTest.java:81–111`; `Util.java:38–108`.
- `org.openmarkov.integrationtests/src/test/.../inference/heuristics/Tools.java:70`
— acceso a `stp.strategyTrees`.
- `org.openmarkov.integrationtests/src/test/.../io/PGMXReadersTest.java:40,43,56,57,65`
— conjunto `POTENTIALS_WITHOUT_PGMX_REPRESENTATION` con clases excluidas; carga dinámica de `SDAGStrategyTree` por nombre.
- `org.openmarkov.integrationtests/.../gui/AllPotentialsHaveAPanel.java:15,21` —
prueba que todo `Potential` descubierto por `PluginSearch` tiene un `PotentialPanel` asociado.

- `org.openmarkov.stochasticPropagationOutput/src/test/.../ExampleNets.java:47-225`
— mismas construcciones que en inference tests.

Estos tests dependen sobre todo de constructores públicos y getters; *no* asumen acceso directo al `double[]` salvo en accesos de lectura.

3.13. Resumen agregado por módulo

La figura 4 resume el grado de acoplamiento por módulo externo, y la tabla 1 cuantifica los puntos calientes.

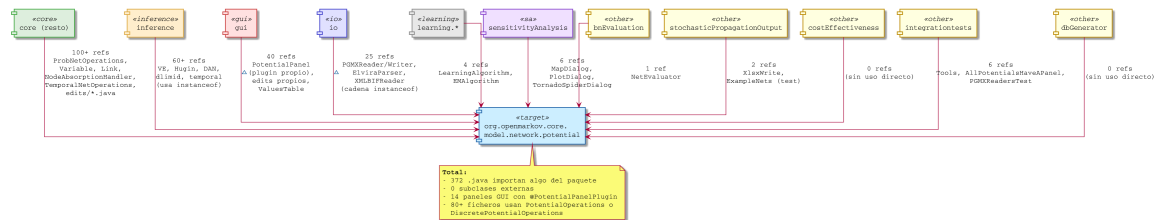


Figura 4: Acoplamiento de módulos externos al paquete `potential`.

Módulo	Refs	Constr.	instanceof	Mutadores	Clientes principales	Acoplamiento dominante
core (resto)	alto	100+	~15	14	ProbNetOps, Variable, edits	profundo (<code>.uncertainValues</code>)
inference	alto	61	~25	4	VE, Hugin, DAN	<code>instanceof</code> + <code>.strategyTrees</code>
gui	alto	16	~30	6	PotentialPanel + edits	cadena <code>instanceof</code> + plugin
io	alto	34	~20	2	PGMX + Elvira	cadena <code>instanceof</code> en writers
learning.core / .algorithm	medio	2	0	0	LearningAlgorithm, EM	<code>getValues()[i]</code>
sensitivityAnalysis	medio	0	0	0	MapDialog, Plot, Tornado	<code>getValues()[i]</code> (lectura)
bnEvaluation	bajo	0	0	0	NetEvaluator	<code>getValues()[0]</code> (lectura)
stochasticPropagationOutput	bajo	2	0	0	XlsxWrite	<code>getValues()[i]</code> (lectura)
costEffectiveness	nulo	0	0	0	—	—
dbGenerator	nulo	0	0	0	—	—
integrationtests	medio	2	~5	0	PGMXReadersTest, Tools	<code>strategyTrees</code> (test)

Tabla 1: Resumen de acoplamiento por módulo. “Refs” es cualitativo (alto/medio/bajo); las columnas numéricas cuentan ficheros distintos en producción.

4. Replanteamiento de las opciones de refactor

A continuación se reformulan las opciones del informe previo a la luz de los datos del análisis cruzado. Cada opción incluye *huella esperada en módulos externos y condicionantes* concretos.

4.1. Opción A — Higiene local

(riesgo bajo)

- Eliminar `operation/Util.java` (formato/paths, no propio del paquete) → mover a `org.openmarkov.java`.
- Borrar ~130 líneas de código comentado en `MaxOutVariable.java:172-304`; comentarios obsoletos en `TablePotential`, `TreeADDPotential`, `Potential`.
- Añadir `hashCode()` consistente con `equals()` en `Potential`, `TablePotential`, `UncertainTablePotential`, `StrategicTablePotential`.
- Hacer null-safe `Potential.equals()` y `TablePotential.equals()`.

- Hacer `Potential.properties` privado con getter/setter (**huella externa nula**: 0 accesos directos).
- Documentar `Potential.setPotentialRole` o restringirlo a `package-private`.

Huella externa estimada: 0 ficheros producción tocados, salvo el movimiento de `Util.java` (huella nula porque sólo se usa internamente, *verificar antes*).

Pros

- Riesgo prácticamente nulo. Las pruebas property-based existentes (memoria del proyecto, plan `project_test_plan.md`) cubren `equals/hashCode` y mutaciones.
- Mejora robustez (NPE en `equals`) sin coste arquitectónico.

Contras

- No resuelve los problemas estructurales (defaults que lanzan, plugins a tres bandas, etc.).
- El cambio de `hashCode` obliga a verificar uso en colecciones `HashMap/HashSet`; es muy improbable que `Potential` se use como clave (no aparecería en mapas) pero es disciplina obligatoria revisarlo.

4.2. Opción B — Completar la migración a interfaces de capacidad

(riesgo medio, recompensa alta)

Eliminar de `Potential` los *defaults* que lanzan `UnsupportedOperationException` y dejar que cada capacidad sea verdaderamente opcional. La forma objetivo es la figura 5.

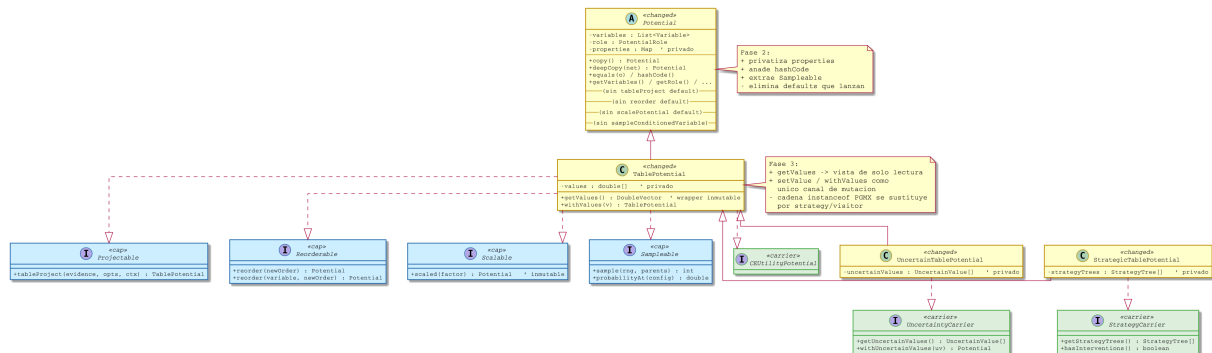


Figura 5: Estado objetivo tras la fase 2: capacidades canónicas, sin *defaults* que lanzan, campos privados y wrappers de mutación explícitos.

Pasos concretos:

1. Promover `Projectable.tableProject` a la única firma oficial. Eliminar `Potential.tableProject(EvidenceCache, InferenceOptions, List<TablePotential>)`. Para cubrir el caso usado por `ProductPotential`, `SumPotential`, `ICIPotential`, introducir `ProjectionContext` que envuelve la lista de potenciales ya proyectados.
2. Crear `Sampleable` con `sampleConditionedVariable` y `probabilityAt`. Mover el default que devuelve 0 fuera de `Potential`.
3. Borrar `Potential.scalePotential` y `Potential.reorder(...)`. Las ~12 implementaciones vacías o que devuelven `copy()` se eliminan.

4. Privatizar `StrategicTablePotential.strategyTrees` y `UncertainTablePotential.uncertainValues`; exponer vía `StrategyCarrier.getStrategyTrees()`, `StrategyCarrier.withStrategyTrees(...)`, equivalentes para `UncertaintyCarrier`.

Huella externa estimada (verificada):

- **2 sitios** de escritura directa a `strategicTablePotential.strategyTrees` en `VEOptimalIntervention.java:85`
- **6 sitios** de lectura/escritura a `uncertainPotential.uncertainValues` en `ProbNetOperations.java (4)` y `SystematicSampling.java (2)`.
- **5 sitios** de tests con escritura literal en `SensitivityAnalysisFactory.java`.
- Inferencia y GUI siguen invocando `tableProject` a través de las firmas existentes.

Total: ~**15 sitios externos a tocar**.

Pros

- Resuelve el problema documentado en `Projectable.java:25`: la base ya no miente sobre las capacidades que ofrece.
- Las property-based ya existentes para `StrategicTablePotential` (memoria del proyecto: 7 tests en `StrategicTablePotentialTest.java`) actúan como red de seguridad.
- No hay subclases externas (sección 3.1), así que ningún plugin de tercero rompe.

Contras

- 15+ sitios producción a tocar, distribuidos entre `core`, `inference` y `tests`. Necesita una *property* previa que ancle el flujo `ProbNetOperations` con incertidumbre antes del cambio.
- Cambio de firma en `Projectable.tableProject` obliga a actualizar 17 clientes externos identificados.

4.3. Opción C — Renombrar `PotentialRole.UNSPECIFIED` a `UTILITY`

(riesgo medio, principalmente PGMX)

Huella externa:

- Lectores/escritores PGMX (`PGMXReader_0_2.java:1429`, `PGMXWriter_0_2.java:746`, `PGMXWriter_1_0`): mapeo `role=utility` ↔ `UNSPECIFIED` debe convertirse en `role=utility` ↔ `UTILITY` con compatibilidad de *lectura* para archivos antiguos (que ya escriben `utility`).
- Cualquier case `UNSPECIFIED` en producción: estimado en ~30 sitios totales (entre el paquete y consumidores directos como `TableProbOperations`, `ProbNetWriter`, `InferenceManager`).

Pros

- Modelo conceptualmente honesto. Reduce comentarios del tipo “esto es utility en realidad”.
- Permite eliminar duplicación con `isAdditive` (que actualmente lo redunda).

Contras

- Compatibilidad de archivos PGMX existentes; si se rompe la lectura, ningún test sobre `*.xml legacy` en `src/test/resources/nets/` pasará.
- Riesgo de regresión silenciosa en flujos de utilidad (donde `UNSPECIFIED` es el marcador).

4.4. Opción D — Aplanar la fachada de operaciones

(riesgo medio, sobre todo *churn*)

Mantener una sola fachada (probablemente `PotentialOperations`) y suprimir `DiscretePotentialOperations` (o viceversa). Reorganizar `operation/` en submódulos temáticos (`arithmetic/`, `elimination/`, `factory/`, `transform/`).

Huella externa: 24 ficheros importan `DiscretePotentialOperations`, 28 importan `PotentialOperations`. Si se conserva `PotentialOperations`, hay que migrar 24 importaciones.

Pros

- Una única fachada elimina la decisión “¿qué fachada uso aquí?”.
- Reorganizar `operation/` reduce la sensación de paquete plano con 15 ficheros heterogéneos.

Contras

- 24+ ficheros con `import` cambiado: ruido en `git blame`.
- Hay que decidir qué hacer con `LinkRestrictionPotentialOperations` (5 clientes externos, de facto público) y `AuxiliaryOperations` (3 clientes): no se pueden ocultar.

4.5. Opción E — Unificar las firmas de `tableProject`

(riesgo medio, queda incluida en la Opción B)

Dejar una única firma `tableProject(EvidenceCase, InferenceOptions, ProjectionContext)`. Sólo `ProductPotential.tableProject:86`, `SumPotential.tableProject:85` y `ICIPotential.tableProject:201` necesitan el contexto. Esto resuelve la duplicación de `Potential.tableProject(...)` con `Projectable.tableProject(...)`.

Huella externa: 17 ficheros fuera del paquete invocan alguna sobrecarga de `tableProject`. La migración es mecánica: añadir `ProjectionContext.empty()` en los call-sites.

4.6. Opción F — Composición sobre herencia (`Strategic` / `Uncertain` como decoradores)

(riesgo alto)

Reformular `StrategicTablePotential` y `UncertainTablePotential` como decoradores (`TablePotential.delegate + state`) en lugar de subclases. Permite combinaciones (`uncertain × strategic`) que hoy son imposibles, citadas como caso “raro no soportado” en `TreeADDPotential.blendPotentials` (líneas 622–624).

Huella externa estimada: 317 ficheros referencian `TablePotential`, `UncertainTablePotential` o `StrategicTablePotential`. La mayoría no necesitan cambio si:

- `TablePotential` mantiene la misma API pública.
- Los decoradores *son* `TablePotential` (vía interfaz, no por herencia).
- Las cadenas `instanceof` en `PGMXWriter_0_2.java:746–860` pasan a usar las interfaces `StrategyCarrier` / `UncertaintyCarrier`.

Aún así, los 17 accesos directos a campos públicos (`strategyTrees`, `uncertainValues`) en código externo ya analizados (sección 3.2) hay que migrarlos.

Pros

- Resuelve el caso “uncertain $\times$ strategic” anotado en `TreeADDPotential.java:622–624`.
- Facilita futuras capacidades opcionales sin explosión de subclases.

Contras

- Cambio profundo. Coste de allocation por decorador.
- Los plugins GUI (`@PotentialPanelPlugin(potentialClasses=...`) están atados por literal de clase; el panel de `StrategicTablePotential` no existe (hereda de `TablePotentialPanel`), pero conviene revisar.

4.7. Opción G — Inmutabilidad estricta

(riesgo muy alto)

Eliminar `setValues`, `setUniform`, `scalePotential`, `setUncertainValues` y todas las mutaciones in-place. Foco principal de mutación según el análisis cruzado:

- 7 escrituras a `getValues()[i]` en producción (sección 3.2).
- 63 invocaciones externas a `setValues`, `setUncertainValues`, `setNoisyParameters`, `setLeakyParameters`, etc. Concentradas en `org.openmarkov.core/.../action/` (ediciones `PNEdit`) y `DANFactory`.
- Edición GUI `TablePotentialValueEdit.java:162` es el único *write-back* desde la GUI.

Pros

- Thread-safety; razonamiento simplificado; cache seguro.
- Previene casos como `TablePotential.tableProject:191–193`, donde se devuelve `this` sin clonar el array (aliasing accidental).

Contras

- Las ediciones `PNEdit` dependen de mutar y revertir in-place. Migrar a “edit + replace potential in node” afecta `ProbNet`, `Node`, `PNESupport` (semántica de undo).
- Coste de allocation puede impactar redes grandes: cada multiplicación ya devuelve nuevas instancias, pero `normalize`, `setUniform` y `scalePotential` optimizan in-place hoy.
- *No es prerequisite* para Opciones A–F. Postergable.

4.8. Opción H — Sustituir cadena `instanceof` de `PGMX` por *strategy/visitor*

(riesgo medio, valor alto)

La cadena `PGMXWriter_0_2.java:746–860` y su gemela en `PGMXWriter_1_0` crecen cada vez que se añade un subtipo. Posibles soluciones:

- *Strategy registrada por Class*, simétrica al mapa `potentialParsers` que ya existe en `PGMXReader_0_2.java:60` para lectura.
- *Visitor* con `Potential.accept(PotentialVisitor)`; es intrusivo en la base.
- *Pattern matching switch* (Java 21+): mantener la cadena pero en una expresión `switch` más limpia, sin cambiar la API.

Huella externa: sólo `org.openmarkov.io`. La estructura de escritura del XML PGMX no cambia.

Pros

- Cada nuevo subtipo registra su escritor en un punto único.
- Simetría con la lectura: ambas direcciones consultan un mapa.

Contras

- `org.openmarkov.io` crece en infraestructura.
- Si se eligen “strategies registradas por `classgraph`”, se añade un cuarto sistema de plugin a los tres ya existentes.

4.9. Comparativa de opciones

Opción	Riesgo	Esfuerzo	Externa	Rev.	Beneficio	Prereq.
A. Higiene local	bajo	1–2 ses.	~0	sí	medio	—
B. Capacidades canónicas	medio	3–4 ses.	~15	sí	alto	A
C. UNSPECIFIED → UTILITY	medio	2 ses.	~30	no [†]	medio	A
D. Aplanar fachadas	medio	2 ses.	24	sí	medio	—
E. Unificar <code>tableProject</code>	medio	1–2 ses.	17	sí	medio	B
F. Composición Strategic / Uncertain	alto	6+ ses.	~25	parcial	alto	B
G. Inmutabilidad estricta	muy alto	10+ ses.	70+	no	alto	B,F
H. Strategy en <code>PGMXWriter</code>	medio	2 ses.	0 prod.	sí	medio	—

Tabla 2: Comparativa. “Externa” = ficheros no-paquete a tocar (estimado). [†] no reversible a nivel de fichero PGMX si rompemos compatibilidad de lectura.

5. Plan de acción por fases

Cinco fases incrementales, pensadas para ser cada una un PR autónomo con verde en `mvn install` y todas las property-based ya existentes.

5.1. Fase 0 — Anclar el comportamiento actual con properties

Objetivo: ampliar la red de tests antes de tocar la base. Se inspira en la fase 0 de `ProbNet refactoring` (memoria del proyecto: `project_probnet_refactoring.md`), que pasó de 37% a ~80% de cobertura antes del refactor.

- Property-based para `Potential.equals/hashCode` (con ejemplos de subtipos heterogéneos).
- Property-based para `deepCopy`: el grafo de variables se reasigna al `copyNet`; ningún campo queda “apuntando” al original.
- Property-based para `DiscretePotentialOperations.multiplyAndMarginalize` con potenciales que mezclan `StrategicTablePotential` y `TablePotential` (intervenciones se preservan).
- Property-based para `ProbNetOperations.proyect-uncertain` (línea 618–634), que es el único punto productivo con escritura externa a `uncertainValues`.

Criterios de aceptación: todas las nuevas properties pasan en la rama actual sin tocar código de producción.

Huella externa: 0.

5.2. Fase 1 — Higiene local (Opción A)

Objetivo: cerrar inconsistencias menores antes de cualquier cambio de API.

- Mover `operation/Util.java` a `org.openmarkov.java`.
- Borrar código comentado en `MaxOutVariable`, `TablePotential`, `Potential`, `TreeADDPotential`.
- Añadir `hashCode()` consistentes y `Objects.equals`-style null-safety.
- Privatizar `Potential.properties`, exponer vía `getter/setter`.
- Auditar `Potential.setPotentialRole` y restringir su visibilidad.

Criterios de aceptación: property-based de la Fase 0 verdes; `mvn install -DskipTests=false` verde; ninguna firma pública alterada salvo `properties` (de campo a `getter` — 0 clientes externos).

Huella externa: 0 modificaciones esperadas.

5.3. Fase 2 — Capacidades canónicas (Opción B + Opción E)

Objetivo: eliminar los *defaults* que lanzan en `Potential`; unificar `tableProject`; privatizar `strategyTrees` y `uncertainValues`. Estado objetivo en figura 5.

Subfases:

- 2a Introducir `ProjectionContext`; migrar las 17 invocaciones externas de `tableProject(...)` a la firma única; eliminar la sobrecarga con `List<TablePotential>`.
- 2b Crear `Sampleable`; migrar `sampleConditionedVariable` y `getProbability`. Borrar el default 0 de `Potential.getProbability`.
- 2c Privatizar `StrategicTablePotential.strategyTrees`. Modificar `VEOptimalIntervention.java:85,86` y `Tools.java:70` (3 sitios).
- 2d Privatizar `UncertainTablePotential.uncertainValues`. Modificar `ProbNetOperations.java:618-634`, `SystematicSampling.java:150,162` y los 5 sitios de tests (`SensitivityAnalysisFactory`). 11 sitios.
- 2e Borrar `Potential.scalePotential`, `Potential.reorder(...)`. Limpiar las ~12 implementaciones vacías.

Criterios de aceptación por subfase: `mvn install` verde, property-based de Fase 0 verdes, tests de inferencia (`InferenceAlgorithmIDTest`, `DANTest`) verdes.

Huella externa total: ~15–20 ficheros producción modificados, distribuidos en `core`, `inference`, `tests`.

5.4. Fase 3 — Strategy en PGMXWriter (Opción H)

Objetivo: eliminar la cadena `instanceof` de `PGMXWriter_0_2.java:746-860` y su gemela en `PGMXWriter_1_0`.

- Crear `PotentialWriterStrategy` con un mapa `Map<Class<? extends Potential>, PotentialWriterStrategy>` simétrico al `potentialParsers` de `PGMXReader_0_2.java:60`.
- Cada subtipo registra su writer mediante el mismo mecanismo (sea `classgraph` o un mapa estático en `PGMXWriter_0_2.buildPotentialWriters()`).
- Conservar la firma pública de `writePotential(...)`.

Criterios de aceptación: todos los XMLs de `src/test/resources/nets/` se leen, se escriben y la diferencia es únicamente formato (no semántica).

Huella externa: sólo `org.openmarkov.io`.

5.5. Fase 4 — Aplanar fachadas (Opción D)

Objetivo: una sola fachada en `operation/`; reorganización en submódulos temáticos.

- Decidir entre `PotentialOperations` y `DiscretePotentialOperations` (probablemente `PotentialOperations` como única, por su nombre genérico).
- Migrar las ~24 importaciones del lado discontinuado.
- Considerar reorganizar `operation/` en `operation/arithmetic/`, `operation/elimination/`, `operation/factory/`, `operation/transform/`. `LinkRestrictionPotentialOperations` y `AuxiliaryOperations` se mantienen como puntos públicos (5 y 3 clientes externos respectivamente).

Criterios de aceptación: sólo cambian imports; nada de lógica.

Huella externa: ~24 ficheros con `import` modificado. Se hace en un único PR para minimizar conflictos.

5.6. Fase 5 — Limpieza semántica de `PotentialRole` (Opción C, opcional)

Objetivo: renombrar `UNSPECIFIED` a `UTILITY`; mantener compatibilidad de lectura PGMX para ficheros existentes.

- Introducir `UTILITY` en `PotentialRole`.
- Cambiar `PGMXReader_0_2.java:1429` para mapear `role=ütility` → `UTILITY`.
- Cambiar `PGMXWriter_0_2.java:746` y `PGMXWriter_1_0` para escribir `role=ütility` desde `UTILITY`.
- Migrar ~30 case `UNSPECIFIED` en producción.
- Eliminar `UNSPECIFIED`.

Criterios de aceptación: todos los `*.xml` de `src/test/resources/nets/` (incluidos legacy) se leen exactamente igual; escritura → relectura es `NET_eq` (igualdad semántica).

Huella externa: ~30 ficheros producción modificados.

5.7. Fases postergables: F (composición) y G (inmutabilidad)

Opción F (composición `Strategic/Uncertain` como decoradores): sólo debe abordarse si aparece un caso de uso concreto que requiera la combinación `uncertain × strategic`, o si se planea introducir nuevas capacidades cruzadas. Hoy es una mejora de elegancia, no de funcionalidad.

Opción G (inmutabilidad estricta): requiere rediseñar el sistema de ediciones `PNEdit` para asumir “edit + replace potential in node” en lugar de “edit + mutate”. Es un proyecto en sí mismo, con impacto en `ProbNet`, `Node`, `PNESupport`, todas las `PNEdit` que mutan y la GUI. **No abordar a corto plazo.**

6. Riesgos y mitigaciones

6.1. Riesgos transversales

- **Compatibilidad PGMX.** Cualquier cambio en `PotentialRole`, en nombres `@PotentialType` o en estructura de subtipos puede romper redes guardadas en disco. Mitigación: property-based de *round-trip* con todos los `*.xml` de `src/test/resources/nets/` antes de cada PR.
- **Plugins GUI atados por literal.** `@PotentialPanelPlugin(potentialClasses=...`) fijan la clase en compilación. La eliminación o renombrado de un `Potential` rompe esto. Mitigación: el test `AllPotentialsHaveAPanel.java` (integración) verifica el invariante; debe quedar verde tras cada fase.
- **Reflexión de `PotentialUtils`.** Cambios de constructor rompen `instanciateSafely`. Mitigación: property-based que invoca `instanciateSafely` sobre todos los `POTENTIALS_BY_NAME`, asegurando éxito.
- **Aliasing en `tableProject`.** `TablePotential.tableProject` línea 191–193 devuelve `this` sin clonar; cambiar esto puede afectar inferencias que dependen de la identidad. Mitigación: property que verifica que multiplicar dos veces el mismo potencial no contamina valores.

6.2. Riesgos por fase

Fase 0: coste cognitivo de escribir tantas properties; *no* hay riesgo en producción.

Fase 1: mínimo. La privatización de properties podría romper algún sitio no detectado; mitigar con `find . -name "*.java" | xargs grep -E ".properties"` dentro de `org.openmarkov.*`.

Fase 2: el cambio de firma de `tableProject` es el más invasivo. Mitigar haciendo 2a en su propio PR antes de las demás subfases.

Fase 3: muy contenido (¡o solo). Verificación con tests de *round-trip*.

Fase 4: sólo imports. Riesgo de conflictos de *merge* con ramas activas; coordinar.

Fase 5: PGMX es el riesgo principal. Property-based de *round-trip* antes y después.

7. Recomendación final

1. **Iniciar por la Fase 0** (anclaje con properties). Es consistente con el modus operandi del proyecto: se hizo en `ProbNet refactoring` y dio frutos.
2. **Ejecutar Fases 1–3 en orden** (higiene local → capacidades canónicas → writer strategy). Cada una es autocontenida y proporciona valor inmediato. Total estimado: 6–8 sesiones.
3. **Fase 4 (aplanar fachadas)** sólo cuando 1–3 estén consolidadas. Es de bajo riesgo pero alto *churn*.
4. **Fase 5 (UTILITY)** si se llega a un consenso sobre compatibilidad PGMX. La compatibilidad de lectura es ineludible.
5. **Fases F y G postergadas.** Reabrir si surge un caso de uso concreto.

El argumento más fuerte a favor de las Fases 1–3 es que *todas las clases que hay que tocar son del workspace propio* (sección 3.1: 0 subclases externas) y los puntos de mutación externa están bien identificados (15–20 ficheros, sección 3.2). Las opciones más agresivas (F, G) requieren un caso de uso que justifique su coste.

A. Comandos usados para el análisis cruzado

Listado abreviado de los comandos `grep`/`find` usados (todos desde `/home/manuel/workspace-openmarkov/`):

```
1 # Subclases externas (estricto)
2 grep -rEn "^[[:space:]]*(public|abstract|final|@w+(\([^)]*\))?\s+)*class[[:space:]]+w
   ↳ +(<[>]+>)?[[:space:]]+extends[[:space:]]+(Potential|TablePotential|...)\b" \
3 --include='*.java' .
4
5 # Accesos externos a campos publicos
6 grep -rEn "\b[w+\.](uncertainValues|strategyTrees)\b" --include='*.java' . \
7 | grep -v target | grep -v "/org/openmarkov/core/model/network/potential/"
8
9 # getValues()[i] externos
10 grep -rEn "getValues\(\)\s*\[" --include='*.java' . \
11 | grep -v target | grep -v "/org/openmarkov/core/model/network/potential/"
12
13 # Imports de internals de operation/
14 grep -rEn "import\s+org\.openmarkov\.core\.model\.network\.potential\.operation\.(
   ↳ TablePotentialArithmetic|...)\b" \
15 --include='*.java' . | grep -v target
16
17 # Cadena instanceof en PGMXWriter
18 grep -nE "instanceof\s+.*w*Potential" \
19 org.openmarkov.io/.../PGMXWriter_0_2.java
20
21 # @PotentialPanelPlugin
22 grep -nE "@PotentialPanelPlugin" \
23 org.openmarkov.gui/src/main/java/org/openmarkov/gui/dialog/common/*.java
```

Listado 4: Comandos clave del análisis cruzado.

B. Glosario

Capacidad (interfaz de capacidad)

Interfaz que declara una operación opcional sobre `Potential` (`Projectable`, `Reorderable`, `Scalable`, `Sampleable`, `StrategyCarrier`, `UncertaintyCarrier`, `CEUtilityPotential`). Sustituye al patrón “método abstracto en la base que las subclases satisfacen lanzando `UnsupportedOperationException`”.

ProjectionContext (propuesto)

Objeto inmutable que envuelve la evidencia, las `InferenceOptions` y la lista de potenciales ya proyectados. Reemplaza la sobrecarga `tableProject(..., List<TablePotential>)`.

Decorador (composición)

Patrón en el que `StrategicTablePotential` y `UncertainTablePotential` envolverían un `TablePotential` en lugar de heredar de él, y anunciarían capacidades adicionales vía las interfaces `StrategyCarrier` / `UncertaintyCarrier`.

Round-trip PGMX

Test que carga un fichero `.xml`, lo escribe a un fichero nuevo y comprueba que la red resultante es semánticamente equivalente a la original.